Лабораторная работа № 2

Архитектура компьютеров

Студент: Мамова Эрланда Тахировна

Группа: НКА-04-25

Содержание

- Лабораторная работа № 2
- Содержание
- Цель работы
- Задание
- Теоретическое введение
- Выполнение лабораторной работы
- Выводы
- Список литературы{.unnumbered}

Цель работы

Изучить идеологию и применение средств контроля версий и освоить умения по работе с git

Задание

Создать базовую конфигурацию для работы с git. Создать ключ SSH. Создать ключ PGP. Настроить подписи git. Зарегистрироваться на Github. Создать локальный каталог для выполнения заданий по предмету.

Теоретическое введение

Системы контроля версий (Version Control System, VCS) применяются при работе нескольких человек над одним проектом. Обычно основное дерево проекта хранится в локальном или удалённом репозитории, к которому настроен доступ для участников проекта. При внесении изменений в содержание проекта система контроля версий позволяет их фиксировать, совмещать изменения, произведённые разными участниками проекта, производить откат к любой более ранней версии проекта, если это требуется.

В классических системах контроля версий используется централизованная модель, предполагающая наличие единого репозитория для хранения файлов. Выполнение большинства функций по управлению версиями осуществляется специальным сервером. Участник проекта (пользователь) перед началом работы посредством определённых команд получает нужную ему версию файлов. После внесения изменений, пользователь размещает новую версию в хранилище. При этом предыдущие версии не удаляются из центрального хранилища и к ним можно вернуться в любой момент. Сервер может сохранять не полную версию изменённых файлов, а производить так называемую дельта-компрессию — сохранять только изменения между последовательными версиями, что позволяет уменьшить объём хранимых данных.

Системы контроля версий поддерживают возможность отслеживания и разрешения конфликтов, которые могут возникнуть при работе нескольких человек над одним файлом. Можно объединить (слить) изменения, сделанные разными участниками (автоматически или вручную), вручную выбрать нужную версию, отменить изменения вовсе или заблокировать файлы для изменения. В зависимости от настроек блокировка не позволяет другим пользователям получить рабочую копию или препятствует изменению рабочей копии файла средствами файловой системы ОС, обеспечивая таким образом, привилегированный доступ только одному пользователю, работающему с файлом.

Системы контроля версий также могут обеспечивать дополнительные, более гибкие функциональные возможности. Например, они могут поддерживать работу с несколькими версиями одного файла, сохраняя общую историю изменений до точки ветвления версий и собственные истории изменений каждой ветви. Кроме того, обычно доступна информация о том, кто из участников, когда и какие изменения вносил. Обычно такого рода информация хранится в журнале изменений, доступ к которому можно ограничить.

В отличие от классических, в распределённых системах контроля версий центральный репозиторий не является обязательным.

Среди классических VCS наиболее известны CVS, Subversion, а среди распределённых — Git, Bazaar, Mercurial. Принципы их работы схожи, отличаются они в основном синтаксисом используемых в работе команд.

Выполнение лабораторной работы

установили git и gh

```
root@etmamova:~# dnf install git
Обновление и загрузка репозиториев:
 Fedora 43 - x86_64 - Updates
Репозитории загружены.
Пакет "git-2.53.0-1.fc43.x86_64" уже установлен.

Нечего делать.
root@etmamova:~# dnf install gh
Обновление и загрузка репозиториев:
Репозитории загружены.
Пакет
Установка:
```

рисунок 1 - установка

Выполнили базовую настройка git Задали имя и email владельца репозитория,настроили utf-8 в выводе сообщений git, задали имя начальной ветки (будем называть её master),параметр autocrlf и safecrlf:

```
etmamova@etmamova:~$ git config --global user.name "Erlanda Mamova"
etmamova@etmamova:~$ git config --global user.email "erlanda.mamova@mail.ru"
etmamova@etmamova:~$ git config --global core.quotepath false
etmamova@etmamova:~$ git config --global init.defaultBranch master
etmamova@etmamova:~$ git config --global core.autocrlf input
etmamova@etmamova:~$ git config --global core.safecrlf warn
etmamova@etmamova:~$ ssh-keygen -t rsa -b 4096
```

рисунок 2 - настройка Создала ключи ssh

рисунок 3 - ключи ssh Создала ключи pgp *рисунок 4 - ключи pgp* вывела список ключей и скопировала отпечаток приватного ключа и скопировала сгенерированный PGP ключ в буфер обмена:

```
etmamova@etmamova:~$ gpg --list-secret-keys --keyid-format LONG
[keyboxd]
---------
sec   rsa4096/397787EB9F270648 2026-03-02 [SC]
      74207099E5026A3F86337CEF397787EB9F270648
uid                 [  абсолютно ] Erlanda <erlanda.mamova@mail.ru>
ssb   rsa4096/FA4C8F66457A9D01 2026-03-02 [E]

etmamova@etmamova:~$ gpg --armor --export 74207099E5026A3F86337CEF397787EB9F270648 | xclip -sel clip
```

рисунок 5 - копия ключа Перешла в настройки GitHub (https://github.com/settings/keys), нажала на кнопку New GPG key и вставила полученный ключ в поле ввода. Настроила автоматические подписи коммитов git

рисунок 6 - подписи коммитов авторизировалась

```
etmamova@etmamova:~$ gh auth login
? Where do you use GitHub? GitHub.com
? What is your preferred protocol for Git operations on this host? SSH
? Upload your SSH public key to your GitHub account? /home/etmamova/.ssh/id_ed25519
? Title for your SSH key: GitHub CLI
? How would you like to authenticate GitHub CLI? Login with a web browser

! First copy your one-time code: BEA7-8CE6
```

рисунок 7 - авторизация создала шаблон рабочего пространства

```
etmamova@etmamova:~$ mkdir -p ~/work/study/2025-2026/"Операционные системы"
etmamova@etmamova:~$ cd ~/work/study/2025-2026/"Операционные системы"
etmamova@etmamova:~/work/study/2025-2026/Операционные системы$ gh repo create study_2025-20
26_os-intro --template=yamadharma/course-directory-student-template --public
✓ Created repository Erlanda4/study_2025-2026_os-intro on github.com
  https://github.com/Erlanda4/study_2025-2026_os-intro
etmamova@etmamova:~/work/study/2025-2026/Операционные системы$ git clone --recursive git@gi
thub.com:<owner>/study_2025-2026_os-intro.git os-intro
```

рисунок 8 - шаблон Удалила лишние файлы и создала необходимые каталоги:

```
etmamova@etmamova:~/work/study/2025-2026/Операционные системы/os-intro$ rm package.json
etmamova@etmamova:~/work/study/2025-2026/Операционные системы/os-intro$ echo os-intro > COURSE
etmamova@etmamova:~/work/study/2025-2026/Операционные системы/os-intro$ make
```

рисунок 9 - создание каталогов Отправила файлы на сервер:

```
etmamova@etmamova:~/work/study/2025-2026/Операционные системы/os-intro$ git add .
warning: добавление встроенного git репозитория: os-intro
hint: You've added another git repository inside your current repository.
hint: Clones of the outer repository will not contain the contents of
hint: the embedded repository and will not know how to obtain it.
hint: If you meant to add a submodule, use:
hint:
hint:   git submodule add <url> os-intro
hint:
hint: If you added this path by mistake, you can remove it from the
hint: index with:
hint:
hint:   git rm --cached os-intro
hint:
hint: See "git help submodule" for more information.
hint: Disable this message with "git config set advice.addEmbeddedRepo false"
etmamova@etmamova:~/work/study/2025-2026/Операционные системы/os-intro$ git commit -am 'feat(main): make course st
ructure'
[master f54645d] feat(main): make course structure
 1 file changed, 1 insertion(+)
 create mode 160000 os-intro
etmamova@etmamova:~/work/study/2025-2026/Операционные системы/os-intro$ git push
Перечисление объектов: 3, готово.
Подсчет объектов: 100% (3/3), готово.
При сжатии изменений используется до 4 потоков
Сжатие объектов: 100% (2/2), готово.
Запись объектов: 100% (2/2), 946 bytes | 946.00 KiB/s, готово.
Total 2 (delta 1), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (1/1), completed with 1 local object.
To github.com:Erlanda4/study_2022-2023_os-intro.git
```

рисунок 10 - отправка файлов

Выводы

Мы изучили идеологию и применение средств контроля версий и освоили умения по работе с git

Контрольные вопросы

1. Что такое системы контроля версий (VCS) и для решения каких задач они предназначаются? Системы контроля версий (VCS) — это программные инструменты, которые отслеживают изменения в файлах и позволяют управлять различными версиями документов, кода и других данных.
2. Объясните понятия VCS: хранилище, commit, история, рабочая копия Хранилище (repository) — центральное место хранения всех файлов проекта и истории их изменений. Может находиться локально (на компьютере) или на сервере. Commit (фиксация) — операция сохранения изменений в хранилище. Каждый коммит содержит: Уникальный идентификатор (хэш) Автора изменений Дату и время Описание изменений (commit message) Снимок состояния файлов История (history) — последовательная цепочка коммитов, показывающая эволюцию проекта. Позволяет просматривать, сравнивать и возвращаться к любым предыдущим состояниям. Рабочая копия (working copy) — локальная версия файлов проекта, с которой непосредственно работает пользователь. Это "песочница", где происходят все изменения до их фиксации в хранилище.
3. Централизованные и децентрализованные VCS Централизованные VCS (CVCS): Единое центральное хранилище на сервере Все разработчики подключаются к нему Для работы нужно подключение к серверу История хранится только на сервере Примеры: Subversion (SVN), CVS, Perforce Децентрализованные VCS (DVCS): Каждый разработчик имеет полную копию хранилища

локально Можно работать без подключения к сети История хранится у каждого разработчика Несколько "центральных" репозиториев Примеры: Git, Mercurial, Bazaar

4. Опишите действия с VCS при единоличной работе с хранилищем. Создать новый репозиторий git init скопировать существующий git clone <url-репозитория> Проверить состояние git status Добавить файлы в коммит git add файл.txt # конкретный файл git add . # все файлы Сохранить с описанием git commit -m "что изменил" Посмотреть историю и найти нужный коммит git log --oneline Вернуться к конкретному коммиту git checkout <хэш-коммита> Вернуться к последней версии git checkout main Отменить изменения в файле (не сохраненные) git restore файл.txt Отменить последний коммит (сохраняя изменения) git reset --soft HEAD~1 Полностью удалить последний коммит git reset --hard HEAD~1
5. Работа с общим хранилищем git clone -скопировать репозиторий получить изменения git pull origin main -забрать с сервера git add . git commit -m "описание" git push origin main -отправить на сервер
6. Основные задачи Git Хранение версий Совместная работа Ветвление и слияние Откат изменений Резервное копирование
7. Создание основного дерева репозитория: git init Получение обновлений (изменений) текущего дерева из центрального репозитория: git pull Отправка всех произведённых изменений локального дерева в центральный репозиторий: git push Просмотр списка изменённых файлов в текущей директории: git status Просмотр текущих изменений: git diff Сохранение текущих изменений: добавить все изменённые и/или созданные файлы и/или каталоги: git add . добавить конкретные изменённые и/или созданные файлы и/или каталоги: git add имена_файлов удалить файл и/или каталог из индекса репозитория (при этом файл и/или каталог остаётся в локальной директории): git rm имена_файлов Сохранение добавленных изменений: сохранить все добавленные изменения и все изменённые файлы: git commit -am 'Описание коммита' сохранить добавленные изменения с внесением комментария через встроенный редактор: git commit создание новой ветки, базирующейся на текущей: git checkout -b имя_ветки переключение на некоторую ветку: git checkout имя_ветки (при переключении на ветку, которой ещё нет в локальном репозитории, она будет создана и связана с удалённой) отправка изменений конкретной ветки в центральный репозиторий: git push origin имя_ветки слияние ветки с текущим деревом: git merge --no-ff имя_ветки Удаление ветки: удаление локальной уже слитой с основным деревом ветки: git branch -d имя_ветки принудительное удаление локальной ветки: git branch -D имя_ветки удаление ветки с центрального репозитория: git push origin :имя_ветки
8. Приведите примеры использования при работе с локальным и удалённым репозиториями. git add index.html git commit -m "добавил страницу" С удаленным: git clone git push origin main git pull origin main
9. Что такое и зачем могут быть нужны ветви (branches)? Ветка (branch) — это отдельная линия разработки, которая позволяет работать над кодом изолированно от основной версии проекта. Ветки нужны для: изоляции новых функций и экспериментов (чтобы не сломать основную версию), параллельной работы команды над разными задачами, быстрого исправления багов без остановки разработки, удобного код-ревью перед добавлением изменений и поддержки разных версий проекта одновременно.
10. Как и зачем можно игнорировать некоторые файлы при commit? Как: создать файл .gitignore и перечислить в нем шаблоны файлов/папок (например, *.log, node_modules/, .env), зачем: чтобы не хранить в репозитории временные файлы, зависимости, пароли и системный мусор — это экономит место, повышает безопасность и сохраняет чистоту проекта.

Список литературы{.unnumbered}